



EXPERIMENT 10

Student Name: Shagun

UID: 24BDA70214

Branch: AIT-CSE

Section/Group: 24AIT_KRG-2

Subject Name: ADBMS

Subject Code: 24CSP-333

Experiment 10 -- To study and implement Transactions, ACID properties of Database

Aim : To study and learn the about the transactions, rollback, savepoints and implement a robust PostgreSQL transaction system for the students where multiple students are inserted in single transaction

Question :10

Design a robust PostgreSQL transaction system for the students table where multiple student records are inserted in a single transaction. If any insert fails due to invalid data, only that insert should be rolled back while preserving the previous successful inserts using savepoints. The system should provide clear messages for both successful and failed insertions, ensuring data integrity and controlled error handling.

SQL Query:

DO \$\$

-- Start a transaction

BEGIN

-- Insert multiple students

INSERT INTO students(name, age, class) VALUES ('Anisha',16,8);

INSERT INTO students(name, age, class) VALUES ('Mayank',19,9);

INSERT INTO students(name, age, class) VALUES ('Anisha',17,8); --ERROR

INSERT INTO students(name, age, class) VALUES ('Mayank',19,9);

-- If all succeed

EXPERIMENT 10

RAISE NOTICE ' Transaction Successfully Done';

EXCEPTION WHEN OTHERS THEN

-- If any insert fails

RAISE NOTICE 'Transaction Failed..! Rolling back changes.';

RAISE; -- this will rollback the entire transaction

END;

\$\$;

```
NOTICE: Transaction Failed..! Rolling back changes.
```

```
ERROR: duplicate key value violates unique constraint "students_name_key"  
Key (name)=(Anisha) already exists.
```

```
SQL state: 23505
```

```
Detail: Key (name)=(Anisha) already exists.
```

```
Context: SQL statement "INSERT INTO students(name, age, class) VALUES ('Anisha',17,8)"  
PL/pgSQL function inline_code_block line 7 at SQL statement
```

Learning Outcome

After completing this experiment, the learner will be able to:

1. **Understand and implement** transactions, rollbacks, savepoints.
2. PL/SQL and error handling with implicit transaction to apply business logic.